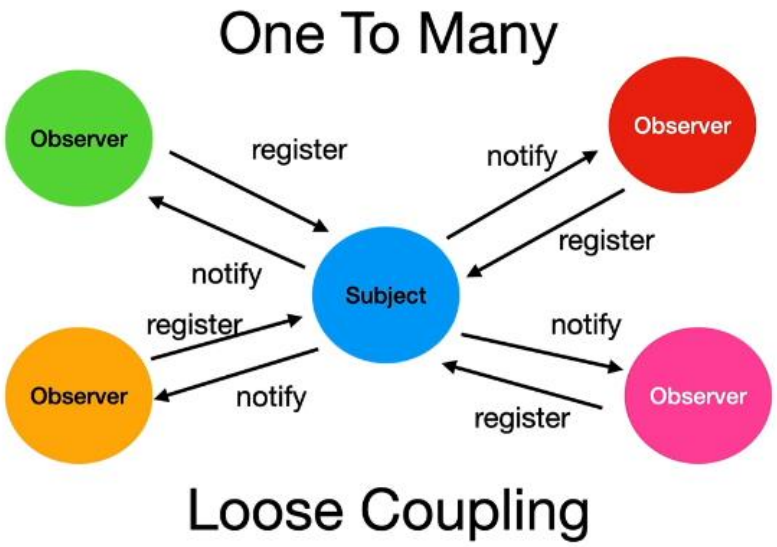
15-Observer Pattern

- Observer Pattern = one object changes → many other objects automatically get notified.
- Observer សង្កេត ពេលមានអ្វីមួយផ្លាស់ប្តូរ វានឹងធ្វើការផ្តល់ពត៌មានអោយទៅកាន់អី ផ្សេងទៀតដែលវាមានទំនាក់ទំនងគ្នា ។

Observer Pattern Behavioural Pattern



Real-Life Uses

Event notification systems

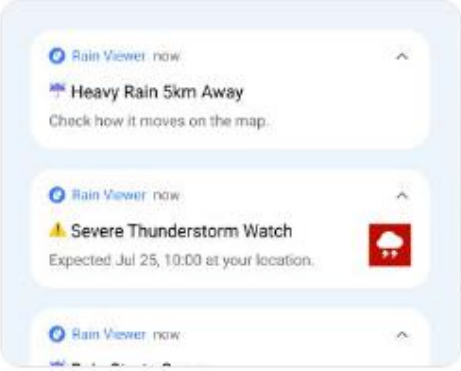
User interface updates

Messaging and notification services

Monitoring and logging systems

Framework-level events

🧩 Real-life example (easy to remember)



Think about:

- You **subscribe to a YouTube channel**
- When the creator uploads a video → **you get notified**

👉 You = *Observer*
👉 YouTube channel = *Subject*

🏗️ Structure (simple terms)

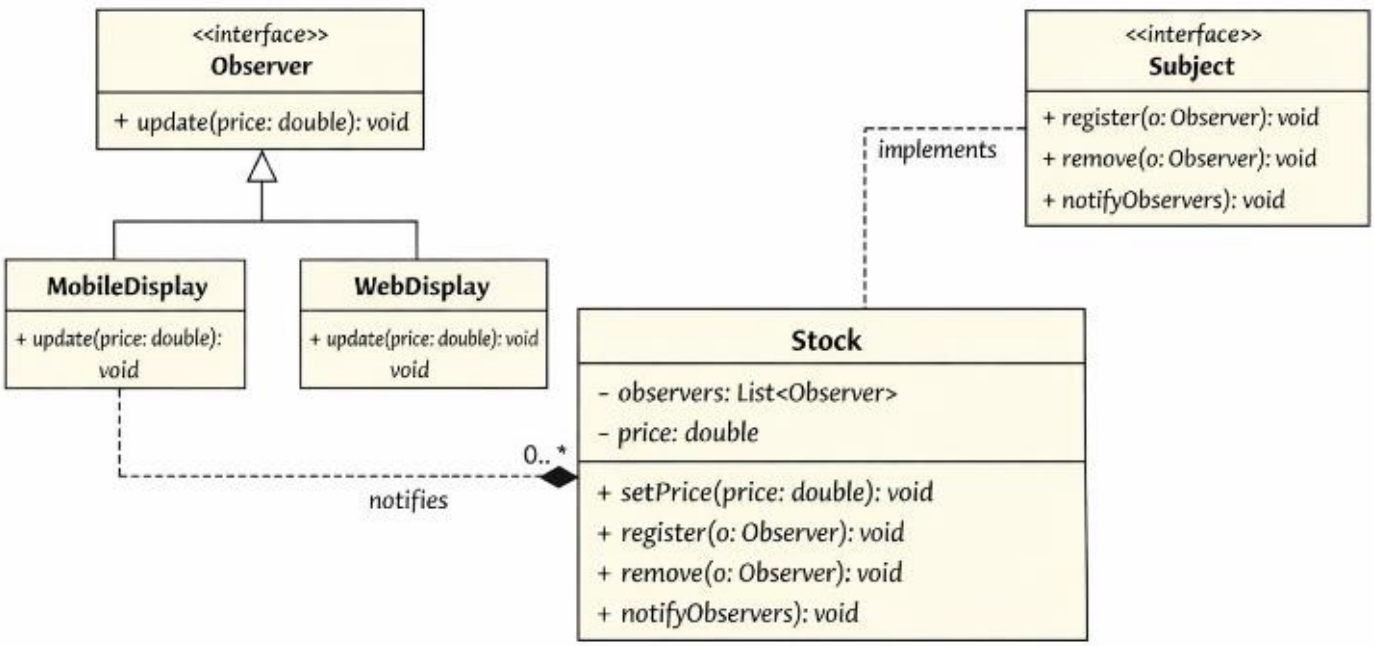
There are 2 main parts:

1. Subject (Publisher)

- Holds a list of observers
- Notifies them when something changes

2. Observer (Subscriber)

- Waits for updates
- Reacts when notified



1-Subject interface

udemy-course-design-pattern ▸ src/main/java ▸ com.vibol.corepattern.obser

```java
package com.vibol.corepattern.observer15;

public interface Subject {

    void register(Observer observer);
    void remove(Observer observer);
    void notifyObservers();
}

```

2-Observer interface

```java
package com.vibol.corepattern.observer15;

public interface Observer {

    void update(double price);

}
```

3- Stock

```java
package com.vibol.corepattern.observer15;

import java.util.ArrayList;

public class Stock implements Subject {

    private List<Observer> observers = new ArrayList<>();
    private double price;

    public void setPrice(double price) {
        this.price = price;
        notifyObservers();
    }

    @Override
    public void register(Observer observer) {
        observers.add(observer);

    }

    @Override
    public void remove(Observer observer) {
        observers.remove(observer);

    }

    @Override
    public void notifyObservers() {
        for( Observer observer: observers) {
            observer.update(price);
        }

    }

}
```

4-MobileDisplay

```java
package com.vibol.corepattern.observer15;

public class MobileDisplay implements Observer {

    @Override
    public void update(double price) {
        System.out.println("Web Display: Stock price update to: " + price);

    }

}
```

5-WebDisplay

```java
package com.vibol.corepattern.observer15;

public class WebDisplay implements Observer {

    @Override
    public void update(double price) {
        System.out.println("Mobile Display: Stock price update to: " + price);

    }
```

6-ObserverTest

```java
package com.vibol.corepattern.observer15;

public class ObserverTest {

    public static void main(String[] args) {
        Stock stock = new Stock();

        stock.register(new WebDisplay());
        stock.register(new MobileDisplay());

        // when the price is update it will call the Web and Mobile to disply

        stock.setPrice(100.6);

    }

}
```

Problems Javadoc Declaration Terminal Console Boot Dashboard

```
<terminated> ObserverTest [Java Application] C:\Program Files\Java\jdk-25\bin\javaw.exe (Apr 27, 2026, 3:23:08 PM – 3:23:08 PM) [pid: 19564]
Mobile Display: Stock price update to: 100.6
Web Display: Stock price update to: 100.6
```

ពេលដែល stock update price the notification will update to the Mobile and Web.